UNIVERSIDADE
EDUARDO
MONDLANE

FACULDADE DE CIÊNCIAS

Departamento de Matemática e Informática

Licenciatura em Informática - Laboral

Trabalho Linguagens de Programação e Compiladores

Linguagem de Programação Erlang

Discentes:

- Joconias Massango
- Liliano Licumba
- Tomás Macuácua

Docente: Alfredo Muchanga

Maputo, Março de 2026

INDICE

1 INTRODUÇÃO 1
2 DESENVOLVIMENTO 2
2.1 História, Motivação e Ambiente de Execução 2
2.1.1 História 2
2.1.2 Motivação 3
2.1.3 Ambiente Básico de Execução 4
2.2 Principais Características 5
2.2.1 Sintaxe 5
2.2.2 Estrutura da linguagem 7
2.2.3 Vantagens e Desvantagens da Linguagem de Programação Erlang 10
2.3 Processo de Compilação em Erlang 11
2.3.1 Natureza da Linguagem Erlang 11
2.3.2 Funcionamento Geral 11
2.3.3 Etapas do Processo de Compilação 12
2.4 Aplicações do Erlang 13
2.4.1 Dominio mais popular 13
2.4.2 Exemplos de Software produzidos com o Erlang 14
3 CONCLUSÃO 16
4 REFERÊNCIAS BIBLIOGRÁFICAS 17

1 INTRODUÇÃO

A crescente demanda por sistemas computacionais capazes de operar de forma contínua, confiável e escalável tem impulsionado o desenvolvimento de linguagens de programação especializadas. Nesse contexto, a linguagem Erlang destaca-se como uma solução robusta para a construção de sistemas distribuídos e concorrentes. Desenvolvida na década de 1980 pela empresa Ericsson, Erlang foi concebida inicialmente para atender às necessidades críticas do setor de telecomunicações, onde falhas e interrupções não são aceitáveis.

Diferentemente de muitas linguagens tradicionais, Erlang adota o paradigma funcional e um modelo de concorrência baseado em processos leves que se comunicam por meio de troca de mensagens. Essa abordagem permite a criação de sistemas altamente resilientes, capazes de continuar operando mesmo diante de falhas parciais. Além disso, a linguagem oferece suporte nativo para distribuição e atualização de código em tempo real, características essenciais em ambientes modernos de alta disponibilidade.

Diante dessas propriedades, Erlang tem sido amplamente utilizada em aplicações que exigem elevado desempenho, tolerância a falhas e capacidade de lidar com grandes volumes de usuários simultâneos. Assim, este trabalho tem como objetivo apresentar as principais aplicações da linguagem Erlang, destacando os seus domínios de uso mais relevantes e exemplos de softwares desenvolvidos com essa tecnologia.

2 DESENVOLVIMENTO

2.1 História, Motivação e Ambiente de Execução

2.1.1 História

A linguagem Erlang foi desenvolvida na década de 1980 por Joe Armstrong, juntamente com Robert Virding e Mike Williams, enquanto trabalhavam na empresa Ericsson.

- **Contexto de Criação**

Na época, a indústria de telecomunicações enfrentava grandes desafios relacionados à confiabilidade dos sistemas. As centrais telefônicas precisavam operar continuamente, com disponibilidade próxima de 100%, e qualquer falha poderia afetar milhões de usuários. As linguagens existentes não ofereciam suporte adequado para concorrência massiva e tolerância a falhas.

- **Desenvolvimento Inicial**

Erlang foi inicialmente concebida como uma linguagem interna da Ericsson, com o objetivo de facilitar o desenvolvimento de sistemas distribuídos. Durante seus primeiros anos, a linguagem evoluiu juntamente com a plataforma OTP (Open Telecom Platform), que fornecia bibliotecas e padrões para desenvolvimento de sistemas confiáveis.

- **Abertura ao Público**

Em 1998, Erlang foi disponibilizada como código aberto, o que permitiu sua adoção fora da Ericsson. Esse marco contribuiu significativamente para sua expansão em áreas como:

- Sistemas de mensagens instantâneas
- Servidores web
- Aplicações financeiras
- Sistemas de jogos online

- **Evolução e Impacto**

Com o tempo, Erlang influenciou diversas outras linguagens e tecnologias modernas voltadas para concorrência e sistemas distribuídos. Sua filosofia de “deixar falhar” (*let it crash*) tornou-se um princípio fundamental no desenvolvimento de sistemas resilientes.

2.1.2 Motivação

A criação do Erlang foi motivada por necessidades práticas extremamente exigentes no contexto de sistemas críticos.

- **Problemas das Linguagens Tradicionais**

As linguagens existentes na época apresentavam limitações como:

- Dificuldade em lidar com múltiplos processos simultâneos
- Alto risco de falhas catastróficas
- Complexidade na implementação de sistemas distribuídos
- Falta de mecanismos eficientes de recuperação de erros

- **Objectivos do Erlang**

Erlang foi projetada para resolver esses problemas com base em quatro pilares principais:

i. **Concorrência Massiva**

Erlang permite a criação de milhões de processos leves (lightweight processes), muito mais eficientes do que threads tradicionais. Esses processos são isolados e executam de forma independente.

ii. **Tolerância a Falhas**

O modelo de supervisão permite que processos sejam monitorados e reiniciados automaticamente em caso de falha. Isso garante a continuidade do sistema

iii. **Distribuição Transparente**

Erlang facilita a comunicação entre processos que podem estar em diferentes máquinas, tornando o desenvolvimento de sistemas distribuídos mais simples.

iv. **Atualização em Tempo Real (Hot Code Swapping)**

Uma característica única do Erlang é a capacidade de atualizar o código de um sistema em execução sem interrompê-lo.

2.1.3 Ambiente Básico de Execução

O ambiente de execução do Erlang é projetado para suportar aplicações concorrentes, distribuídas e tolerantes a falhas. Para isso, ele oferece um conjunto integrado de ferramentas que permitem ao programador desenvolver, testar e executar sistemas de forma eficiente e segura.

- **Plataforma Erlang/OTP**

O principal componente do ambiente é a plataforma Erlang/OTP (Open Telecom Platform), que constitui o núcleo do ecossistema Erlang.

Esta plataforma não é apenas um compilador ou interpretador, mas sim um conjunto completo de ferramentas e bibliotecas que incluem:

  - **BEAM (Erlang Virtual Machine)**: máquina virtual responsável pela execução dos programas, gerenciamento de memória e escalonamento de processos.
  - **Bibliotecas padrão**: funções prontas para manipulação de dados, entrada/saída, redes e sistemas distribuídos.
  - **Framework OTP**: fornece padrões de projeto como supervisores, servidores genéricos (GenServer) e máquinas de estado.
  - **Ferramentas de teste e depuração**: permitem validar e monitorar o comportamento das aplicações.

- **Shell Interativo (Erlang Shell)**

O **Erlang Shell** é o principal ambiente de interação com a linguagem. Ele é iniciado com o comando: `erl.`

- **Ferramentas de Desenvolvimento**

Para melhorar a produtividade, diversas ferramentas podem ser utilizadas em conjunto com Erlang:

  - Editor de código: Visual Studio Code
  - Gerenciador de projetos: **rebar3**

- Controle de versão: Git
- Ferramentas de build e automação

2.2 Principais Características

2.2.1 Sintaxe

A sintaxe de Erlang é bastante distinta de linguagens imperativas como Java ou Python, pois a linguagem é **funcional**. Isso significa que **funções são elementos centrais**, e não há variáveis mutáveis como em linguagens tradicionais.

i. **Declaração de Funções**

Em Erlang, funções são definidas com o operador **->** e terminam com ponto **.**. Funções podem ter **vários padrões de entrada**, permitindo o uso de *pattern matching*.

```
% Função simples que soma dois números

soma(X, Y) -> X + Y.

% Função recursiva usando pattern matching

fatorial(0) -> 1;

fatorial(N) -> N * fatorial(N-1).
```

ii. **Variáveis**

- Variáveis só podem ser atribuídos uma vez (imutabilidade)
- Iniciam com letra maiúscula
- Uma vez atribuídas, não podem ser modificadas, novos valores exigem uma nova variável

```
X = 10,

Y = X + 5.  % Y = 15
```

iii. **Tipos de Dados Comuns**

- **Inteiros e floats:** 10, 3.14
- **Átomos:** constantes simbólicas, começam com minúscula ou entre as simples **`'atom'`**
- **Listas:** [1,2,3]
- **Tuplas:** {ok, 200}
- **Mapas:** %{chave => valor}

```
Lista = [1,2,3],
Tupla = {ok, 200},
Mapa = #{nome => "Tomás", idade => 21}.
```

iv. **Estruturas de Controle**

Erlang não possui loops tradicionais (`for, while`). Em vez disso, usa **recursão** e expressões como `if, case, receive.`

- `if:`

```
if
    X > 0 -> positivo;
    X < 0 -> negativo;
    true -> zero
end.
```

- `Case:` compara valores e executa blocos específicos.

```
case Valor of
    1 -> um;
    2 -> dois;
    _ -> outro
end.
```

- `Receive:` usado para receber mensagens em processos concorrentes.

```
receive
    {Pid, Msg} -> io:format("Recebido: ~s~n", [Msg])
end.
```

v. **Pattern Matching**

Pattern matching é o mecanismo central de Erlang. O operador = não é atribuição, mas unificação: o lado esquerdo é um padrão e o lado direito é um valor. Se o padrão não corresponde ao valor, uma exceção é lançada:

vi. **Comentários**

- Comentários de uma linha: `% comentário`
- Não existem comentários de múltiplas linhas, mas é possível usar múltiplas linhas com `%` consecutivos.

```
% Este é um comentário de uma linha

% Pode continuar na linha seguinte
```

2.2.2 Estrutura da linguagem

A estrutura da linguagem Erlang é projetada para suportar **sistemas concorrentes, distribuídos e tolerantes a falhas**. Diferente de linguagens tradicionais, Erlang organiza seus programas com base em **módulos, funções e processos leves**, seguindo o paradigma funcional.

- **Organização em Módulos**

Em Erlang, os programas são organizados em **módulos**, que correspondem a arquivos com extensão `.erl`. Cada módulo contém um conjunto de funções e define explicitamente quais delas podem ser acessadas externamente por meio da diretiva `-export`.

Essa abordagem promove encapsulamento e modularidade, permitindo que sistemas complexos sejam divididos em partes menores e reutilizáveis. Diferente de linguagens orientadas a objetos, Erlang não utiliza classes, sendo a organização baseada exclusivamente em funções agrupadas em módulos.

```
-module(calculos).

-export([soma/2, fatorial/1]).
```

- `module(nome). -` define o nome do módulo
- `export([função/aritdade]).` – indica quais funções são públicas
- A **aritdade** representa o número de parâmetros da função

- **Paradigma de Programação**

Erlang segue predominantemente o **paradigma funcional**, o que significa que:

- As funções são os principais blocos de construção;
- Os dados são imutáveis;
- O uso de efeitos colaterais é minimizado.

Esse paradigma contribui para maior previsibilidade e segurança, especialmente em ambientes concorrentes, onde múltiplos processos são executados simultaneamente.

- **Componentes Principais da Linguagem**

A estrutura de Erlang baseia-se em três componentes fundamentais:

- **Funções:** utilizadas para definir a lógica do programa;
- **Processos:** unidades independentes de execução;
- **Mensagens:** mecanismo de comunicação entre processos.

Esses elementos substituem conceitos tradicionais como objetos e threads, oferecendo uma abordagem mais segura para concorrência.

- **Modelo de Execução**

Erlang utiliza um modelo de execução **concorrente e assíncrono**, baseado no conceito de **processos leves**. Cada processo é independente, possui sua própria memória e é gerenciado pela máquina virtual da linguagem.

A criação de processos é simples e eficiente, permitindo a execução de milhares ou até milhões de processos simultaneamente. Esse modelo evita problemas comuns como bloqueios e conflitos de memória.

- **Comunicação e Interação**

A interação entre componentes em Erlang ocorre por meio de **troca de mensagens**. Não há compartilhamento de memória entre processos, o que elimina riscos de inconsistência de dados.

Cada processo possui uma fila de mensagens (mailbox), onde recebe e processa comunicações de outros processos. Esse modelo favorece a construção de sistemas distribuídos e altamente escaláveis.

- **Pattern Matching**

Pattern matching é o mecanismo central de Erlang. O operador = não é atribuição, mas unificação: o lado esquerdo é um padrão e o lado direito é um valor. Se o padrão não corresponde ao valor, uma exceção é lançada:

```
% Ligação simples
```

```
X = 42.

% Verificação de estrutura

{ok, Valor} = {ok, 100}.

% Valor = 100

% Falha de pattern matching

{ok, V} = {error, timeout}.

% ** exception error: no match of right hand side value {error,timeout}

% Ignorar parte do padrão com _

{_, Segundo, _} = {a, b, c}.
```

```
% Segundo = b
```

- **Pattern Matching em Cabeçalhos de Função**

Funções em Erlang podem ter múltiplas cláusulas com padrões diferentes. O compilador escolhe a primeira cláusula cujo padrão unifica com os argumentos:

```
-module(formas).
```

```
-export([area/1]).

% Área de diferentes formas geométricas

area({circulo, R}) ->

    math:pi() * R * R;

area({retangulo, L, A}) ->

    L * A;

area({triangulo, B, H}) ->

    (B * H) / 2.

% Uso:

% formas:area({circulo, 5}).       => 78.53...

% formas:area({retangulo, 4, 6}).  => 24
```

```
% formas:area({triangulo, 3, 8}).  => 12.0
```

2.2.3 Vantagens e Desvantagens da Linguagem de Programação Erlang

A avaliação de uma linguagem de programação envolve a análise de seus pontos fortes e limitações, considerando aspectos como desempenho, facilidade de uso, escalabilidade e aplicabilidade. No caso de Erlang, suas características foram projetadas para atender sistemas distribuídos e altamente concorrentes, o que influencia diretamente suas vantagens e desvantagens.

2.2.3.1 Vantagens da Linguagem de Programação Erlang

- **Concorrência -** A linguagem utiliza processos leves e independentes, permitindo a execução simultânea de milhares de processos sem comprometer o desempenho**.**
- **Tolerância a falhas -** Erlang foi projetada com o princípio *"let it crash"*, permitindo que processos falhem sem afetar o sistema como um todo**.**

- **Atualização dinâmica de código –** permite modificar partes do sistema sem interromper sua execução.
- **Escalabilidade** - capaz de operar eficientemente tanto em sistemas pequenos quanto em ambientes distribuídos com múltiplos servidores**.**

2.2.3.2 Desvantagens da Linguagem de Programação Erlang

- **Curva de aprendizado elevada** - principalmente para programadores acostumados com paradigmas imperativos ou orientados a objetos. Conceitos como imutabilidade, recursão e concorrência baseada em mensagens exigem adaptação.
- **Sintaxe pouco intuitiva** - A ausência de estruturas comuns em outras linguagens, como loops tradicionais e classes, pode dificultar a compreensão para iniciantes.
- **Comunidade relativamente menor** - isso implica menor disponibilidade de bibliotecas, frameworks e suporte, o que pode impactar o desenvolvimento de certos tipos de aplicações.
- **Limitações em tarefas de processamento intensivo** - como cálculos matemáticos complexos ou aplicações que exigem alto desempenho computacional.

2.3 Processo de Compilação em Erlang

2.3.1 Natureza da Linguagem Erlang

Erlang é classificada como uma linguagem compilada, pois o seu código fonte não é executado diretamente. Em vez disso, passa por um processo de tradução para bytecode, que é posteriormente executado pela máquina virtual BEAM (Bogdan/Björn's Erlang Abstract Machine). Esse modelo permite independência de plataforma e facilita a execução de programas em diferentes sistemas sem necessidade de recompilação para cada arquitetura.

2.3.2 Funcionamento Geral

O funcionamento do Erlang está baseado na separação entre código fonte, código compilado e ambiente de execução. O programador escreve módulos em arquivos **.erl**,

que são compilados em arquivos **.beam**. Esses arquivos são então carregados pela máquina virtual BEAM, responsável pela execução do programa. Durante a execução, a BEAM gerencia recursos como processos leves, comunicação por mensagens e memória, garantindo estabilidade e eficiência.

2.3.3 Etapas do Processo de Compilação

O processo de compilação em Erlang ocorre de forma estruturada, passando por diferentes fases que transformam o código fonte em um formato executável.

- **Análise léxica**

Nesta etapa inicial, o código fonte é examinado e dividido em unidades básicas chamadas **tokens**, como identificadores, operadores e palavras-chave.

- **Análise sintática**

Os **tokens** gerados são organizados de acordo com as regras gramaticais da linguagem, formando uma estrutura sintática válida.

- **Análise semântica**

Nesta fase, são verificadas as relações entre os elementos do programa, assegurando que o código possui coerência lógica e respeita as regras da linguagem.

- **Geração de representação intermediária**

O código é convertido para uma forma intermediária interna, que facilita otimizações e preparação para execução.

- **Geração de bytecode**

A representação intermediária é transformada em bytecode, armazenado em arquivos com extensão **.beam**.

- **Carregamento**

O bytecode gerado é carregado pela máquina virtual BEAM, tornando o módulo disponível para execução.

- **Execução**

A BEAM executa o bytecode, gerenciando os processos e a comunicação entre eles dentro do sistema.

Fluxo Resumido

Código fonte (.erl)

↓

Análise léxica

↓

Análise sintática

↓

Análise semântica

↓

Representação intermediária

↓

Bytecode (.beam)

↓

Máquina virtual BEAM

↓

Execução

2.4 Aplicações do Erlang

2.4.1 Dominio mais popular

Erlang é amplamente utilizado em sistemas distribuídos, concorrentes e com alta disponibilidade. Seu modelo de execução baseado em processos leves, comunicação assíncrona e tolerância a falhas nativa o torna ideal para cenários onde a confiabilidade e o tempo real são cruciais.

O domínio mais popular de aplicação da linguagem Erlang é o setor de **telecomunicações e sistemas distribuídos em tempo real**.

Erlang foi inicialmente criada para resolver problemas específicos de sistemas de telecomunicações, como:

- Alta concorrência (milhares ou milhões de processos simultâneos)
- Tolerância a falhas (sistemas que não podem parar)
- Distribuição (execução em múltiplos servidores)
- Atualizações sem interrupção (hot code swapping)

2.4.1.1 Principais áreas de aplicação

i. **Telecomunicações**
   - Centrais telefónicas
   - Sistemas de mensagens
   - Redes móveis

ii. **Sistemas distribuídos**
   - Aplicações que funcionam em múltiplos servidores
   - Sistemas de cloud computing

iii. **Sistemas em tempo real**
   - Aplicações que exigem resposta imediata
   - Monitoramento de sistemas críticos

iv. **Backends de alta escalabilidade**
   - Servidores que suportam milhões de utilizadores simultâneos

2.4.2 Exemplos de Software produzidos com o Erlang

Diversos sistemas importantes e de grande escala foram desenvolvidos usando Erlang. Abaixo estão alguns exemplos relevantes:

1. WhatsApp

- Utiliza Erlang no backend para suportar milhões de utilizadores simultâneos
- Consegue lidar com alta concorrência com poucos servidores
- Exemplo clássico de escalabilidade eficiente

2. RabbitMQ

- Sistema de mensageria (message broker)
- Usado para comunicação entre sistemas distribuídos
- Muito popular em arquiteturas de microserviços

3. CouchDB

- Banco de dados NoSQL
- Focado em replicação e distribuição de dados
- Usa JSON para armazenamento

4. Ericsson AXD301

- Sistema de comutação telefónica
- Um dos primeiros sistemas comerciais escritos em Erlang
- Alta confiabilidade em telecomunicações

5. Discord (parcialmente)

- Algumas partes da infraestrutura usam Erlang
- Especialmente para lidar com comunicação em tempo real

6. Klarna

- Plataforma de pagamentos online
- Usa Erlang para garantir disponibilidade e segurança

3 CONCLUSÃO

A linguagem Erlang destaca-se como uma ferramenta poderosa para o desenvolvimento de sistemas distribuídos, concorrentes e altamente confiáveis. O seu uso é especialmente relevante em áreas onde a falha não é uma opção, como telecomunicações e serviços online de grande escala.

Apesar de não ser tão popular quanto outras linguagens modernas, Erlang continua a ser uma escolha estratégica para aplicações críticas que exigem estabilidade, escalabilidade e desempenho contínuo.

4 REFERÊNCIAS BIBLIOGRÁFICAS

- ✓ ARMSTRONG, Joe. Programming Erlang: Software for a Concurrent World. 2. ed. Pragmatic Bookshelf, 2013.
- ✓ CESARINI, Francesco; THOMPSON, Simon. Erlang Programming: A Concurrent Approach to Software Development. O'Reilly Media, 2009.
- ✓ LOGAN, Martin; MERRITT, Eric; CARLSSON, Richard. Erlang and OTP in Action. Manning Publications, 2010.
- ✓ HEBERT, Fred. Learn You Some Erlang for Great Good! No Starch Press, 2013. Disponível em: https://learnyousomeerlang.com
- ✓ ERLANG/OTP Documentation. Ericsson AB, 2024. Disponível em: https://www.erlang.org/doc/
- ✓ VIRDING, Robert; WILLIAMS, Mike; ARMSTRONG, Joe. Concurrent Programming in Erlang. 2. ed. Prentice-Hall, 1996.